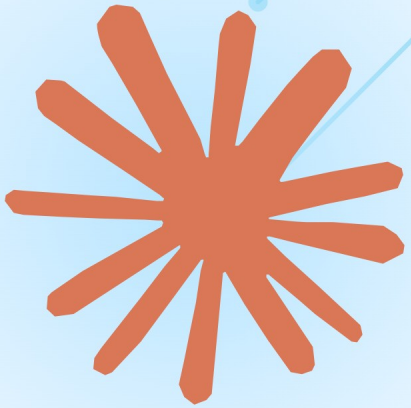
Claude ANTHROP\C

PROGRAMA DE CAPACITACIÓN

Desarrollando con IA

Unidad 6 · Calidad, pruebas y CI/CD

Día 3 · Bloque tarde · Instructor: Felipe Cuevas



felipecuevas.dev

FUNCIONA… EN TU MÁQUINA

ArriendoWeb ya calcula bien. ¿Y cuando nadie mira?

U1 · U2 · U3 · U4 · U5 · **U6 · ahora** · U7 · U8 · U9 · U10

Nivel 55 → 70

Viernes 18:40

Alguien toca el redondeo sin correr los tests.

Fin de semana

Tarifas mal cobradas. El sistema no avisa.

Lunes 09:15

Un reclamo y un log de 900 líneas.

Hoy: ningún cambio llega a main sin pruebas automáticas, y Claude ayuda a leer la falla.

HOJA DE RUTA

Cuatro ideas para hoy

C1

Tests con criterio

Si no puede fallar, no prueba nada.

C2

Cobertura ≠ calidad

Se mide, no se persigue.

C3

Pipeline seguro

Cada push se prueba solo, sin exponer secretos.

C4

Claude en CI

Claude diagnostica; tú corriges y haces merge.

Material oficial: Common workflows («Work with tests»), Best practices, docs de Claude Code GitHub Actions y Claude Code in action.

CONCEPTO 1 · 15 MIN

Tests con criterio

01

Si no puede fallar, no prueba nada.

- Pirámide de tests
- Bug: primero el test
- El test mentiroso

La pirámide: qué delegar y qué no



Delegas a la IA

Unitarios repetitivos y casos borde.

Decides tú

Qué comportamiento importa y qué riesgo vale la pena cubrir.

C1 · TESTS CON CRITERIO

Para corregir un bug, primero el test

1
Reproducir
La entrada exacta que falla.

→

2
Test en rojo
Demuestra el defecto.

→

3
Corregir
El cambio mínimo.

→

4
Verde
Y el test se queda.

PROMPT PARA CLAUDE CODE

```
Escribe un test xUnit que reproduzca el defecto de TarifaBase.
No corrijas nada: muéstrame que falla.
```

DEMO 1 · 6 MIN

El test mentiroso

arriendo-web · rama main

1 · Claude Code propone

«Propón tests de borde para `TarifaBase` que el golden master no tenga»

2 · Lo rompo yo

Cambio **FactorTemporada** a mano en Rider.

3 · Miro los tests

El que sigue en verde es el mentiroso.

Si no puede fallar, no prueba nada. Para saberlo, rómpelo tú, a mano.

CONCEPTO 2 · 10 MIN

Cobertura ≠ calidad

02

Se mide, no se persigue.

Medir en .NET

Leer el reporte

100 %, sin verificar

C2 · COBERTURA ≠ CALIDAD

Qué dice (y qué no) la cobertura



```
dotnet test --collect:"XPlat Code Coverage"
→ coverage.cobertura.xml
```

✓ Te dice

Qué líneas y ramas **nunca se ejecutaron** en los tests.

⚠ No te dice

Si lo que se ejecutó **está bien**. Un test sin Assert también cubre.

Claude te lo lee

Pídele que resuma el XML: qué quedó sin ejecutar y dónde mirar.

Es un mapa, no una meta: muestra dónde mirar.

DEMO 2 · 5 MIN

Cobertura 100 %, verificación 0

arriendo-web · rama main

SOLO CON EL TEST MENTIROSO

100 %

cobertura de **TarifaBase**

1. Claude suma **coverlet.collector** al proyecto de tests.
2. Corre **dotnet test** con cobertura y genera el XML.
3. Claude lee el XML y te lo explica.

Se ejecutó todo. No se verificó nada.

CONCEPTO 3 · 20 MIN

Pipeline seguro

03

Cada push se prueba solo, sin exponer secretos.

GitHub Actions

6 reglas de seguridad

Mi primer workflow

Anatomía de un workflow



on:

Qué lo dispara

`push · pull_request`

jobs: build-test

`runs-on: ubuntu-latest`

`steps:`

1 · checkout del código
2 · instalar .NET
3 · dotnet build
4 · dotnet test



Runner

Máquina limpia de GitHub.

Repos públicos: gratis en runners estándar.

Fuente: GitHub Docs (Billing de Actions), consultado el 10-nov-2026

C3 · PIPELINE SEGURO

Seguridad mínima: 6 reglas

Secretos aparte

Settings › Secrets. Nunca en el código ni en el YAML.

Permisos mínimos

Declara **permissions:** y parte con **contents: read**.

Versión fija

actions/checkout@v6, no @main. Más estricto: el SHA completo.

Logs limpios

Nunca hagas echo de un secreto, ni para depurar.

Forks sin secretos

Un PR desde un fork no recibe tus secretos.

Con límite

timeout-minutes: ningún job corre para siempre.

DEMO 3 · 7 MIN

Mi primer workflow

```yaml
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
  pull_request:
permissions:
  contents: read
jobs:
  build-test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-dotnet@v5
        with: { dotnet-version: '10.0.x' }
      - run: dotnet build
      - run: dotnet test --no-build
```

arriendo-web · rama main

on
Cada push a main y cada PR.

permissions
Solo lectura. Sin secretos.

timeout-minutes · @v6
Con límite y versión fija.

Push a arriendo-web (público) → verde en Actions

EJERCICIO · C3

Encuentra el error

⚠ **Así no**

```
- uses: anthropics/claude-code-action@v1
  with:
    claude_code_oauth_token: "pega-aquí-tu-token"
- run: echo "Token: ..."
```

✓ **Así sí**

```
- uses: anthropics/claude-code-action@v1
  with:
    claude_code_oauth_token: ${{ secrets.CLAUDE_CODE_OAUTH_TOKEN }}
# sin echo del secreto
```

🔑 **¿Ya lo subiste? Genera uno nuevo y borra el secreto: git no olvida.**

CONCEPTO 4 · 25 MIN

Claude en CI

04

Claude diagnostica; tú corriges y haces merge.

- claude-code-action
- Headless
- Tu token y costos
- Otras IA

claude-code-action@v1: dos modos

Interactivo

El del taller

Sin **prompt** en el workflow. Espera un **@claude** en un issue o PR.

```
@claude ¿por qué falla este job?
```

Responde con un comentario en el mismo hilo.

Automatización

Con **prompt**: corre solo ante un evento (PR abierto, horario).

```
prompt: "Resume los PR abiertos"
```

El resultado va al log, salvo que le pidas comentar.

Solo responde a quien tiene escritura en el repo.

Instalación rápida: /install-github-app (admin del repo + gh auth login)

Tu plan B local: claude -p



```
$ dotnet test 2>&1 | claude -p "Explica en 3 líneas por qué falla"
```

Sin interfaz

Una pregunta, una respuesta, y termina.

En tu terminal

Si el CI o la red fallan, lees la falla en tu equipo.

Gasta tu cuota Pro

Usa tu misma sesión de Claude Code.

C4 · CLAUDE EN CI · CREDENCIAL

Tu credencial: el token de tu Claude Pro

1 En tu equipo, una vez: `claude setup-token`

2 Settings › Secrets › Actions: `CLAUDE_CODE_OAUTH_TOKEN`

3 claude.yml lo usa así: `${{ secrets.CLAUDE_CODE_OAUTH_TOKEN }}`

⚠ **Gasta tu cuota y nunca va en un archivo.**

Docs oficiales Claude Code (GitHub Actions) · consultado el 10-nov-2026

Tres frenos para tu claude.yml

```yaml
concurrency:
  group: claude-${{ github.event.issue.number }}
jobs:
  claude:
    timeout-minutes: 10
    steps:
      - uses: anthropics/claude-code-action@v1
        with:
          claude_args: '--max-turns 5 --disallowedTools "Edit,Write"'
```

5 vueltas como máximo **--max-turns**

10 minutos y se corta **timeout-minutes**

1 ejecución a la vez por issue **concurrency**

Con Pro, el costo es tu cuota. Pedidos @claude concretos: menos vueltas.

Actions: gratis en repos públicos · GitHub Docs, consultado el 10-nov-2026

LA REGLA DEL CURSO

Claude diagnostica, tú corriges y haces merge con el CI en verde.



CAPA 1 · WORKFLOW

ci.yml con **contents: read**.

CAPA 2 · HERRAMIENTAS

--disallowedTools "Edit,Write" y **--max-turns 5**.

CAPA 3 · PERSONAS

Merge solo con el CI en verde.

DEMO 4 · 8 MIN

@claude, ¿por qué falla?

arriendo-web · rama del PR

⚠ CI / build-test — **falló** · solo A08 en rojo

Instructor `@claude ¿por qué falla? Solo diagnostica, no hagas commits`

Claude Falla solo A08: el redondeo volvió a **AwayFromZero** en **TarifaBase**. Propuesta: restaurar el redondeo de la U5. Revisa si el cambio era intencional.

Tú corriges → CI en verde → **Merge**

Mismo patrón, otros agentes

Herramienta	Cómo se activa	Dónde corre	Credencial
claude-code-action	@claude o un prompt	Tu workflow	**Token de tu plan (Pro)**
Copilot coding agent	Asignar un issue; @copilot en el PR	Entorno propio sobre GitHub Actions	Plan Copilot Pro, Pro+, Business o Enterprise
openai/codex-action	Un prompt ante un evento (p. ej. PR abierto)	Tu workflow	OPENAI_API_KEY
run-gemini-cli	@gemini-cli, /review, /triage	Tu workflow	GEMINI_API_KEY, Vertex AI u OIDC

El patrón es el mismo: el agente corre en CI con los secretos y permisos que tú defines.

REPASO

Antes del receso: 4 preguntas

C1 TESTS CON CRITERIO

¿Cómo sabes que un test no es mentiroso?

C2 COBERTURA ≠ CALIDAD

Si la cobertura dice 100 %, ¿ya está todo probado?

C3 PIPELINE SEGURO

¿Dónde va el token de Claude y con qué permiso parte el ci.yml?

C4 CLAUDE EN CI

Si @claude propone un arreglo, ¿quién hace el merge?

PARA INVESTIGAR POR TU CUENTA

Para profundizar

Tests e2e

Playwright sobre el portal web.

Mutation testing

Stryker.NET: ¿tus tests detectan cambios?

Otros CI

GitLab CI/CD y GitHub Enterprise Server.

Evals en CI

Un play del AI-native SDLC.

Routines y Agent SDK

Tareas programadas y agentes propios.

Sin secretos

Workload identity federation (OIDC).

15′

Receso

Al volver: trae lo nuevo de la U6 a tu repo pidiéndole ayuda a Claude Code (README de unidad-6/taller/pedidos-web, camino A).

TALLER · 80 MIN · INDIVIDUAL · TU PEDIDOS-WEB

Que ningún cambio llegue a main sin pruebas

5′ Preparación → **20′** **1** Tests que sí prueban → **25′** **2** Pipeline seguro → **25′** **3** La falla → **5′** Cierre

Entregable

- ci.yml en verde
- claude.yml
- Tests revisados
- PR con diagnóstico de @claude resuelto
- docs/calidad/registro-u6.md

Descarga la guía del taller (PDF)

ffelipecuevasc.github.io/
curso-ia-desarrolladores → U6

Licencia: Claude Pro + Claude Code · GitHub, repo público

Mañana · Día 4

Unidad 7 · Code review y colaboración



U1 U2 U3 U4 U5 U6 **U7 · mañana** U8 U9 U10

Tu pipeline y tu token se reutilizan: mañana Claude revisa los PR y el equipo comparte las mismas reglas.

ffelipecuevasc.github.io/curso-ia-desarrolladores